

```

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.optimize import curve_fit
from pathlib import Path

BASE_DIR = Path(__file__).parent

DATA_PATH = BASE_DIR / "FP66-quant" / "2026-07-20"
DATA_KORR_PATH = DATA_PATH / "KorrelationsMessungen"
DATA_RATEN_PATH = DATA_PATH / "RatenMessungen"
DATA_BIT_PATH = DATA_PATH / "BitMessungen"

IMAGE_PATH = BASE_DIR / "Images"
EINZELZ_PATH = IMAGE_PATH / 'Einzelzählraten'
KOINZIDENZ_PATH = IMAGE_PATH / 'Koinzidenz'

### ---- Utility Functions ---- ###
#
ROUND_TO = 3
def get_polaristaoinsgrad(df, key):
    N_max = np.max(df[key])
    N_min = np.min(df[key])
    return (N_max - N_min) / (N_max + N_min)

def get_correlation(df: pd.DataFrame):
    deg_alice = np.array([df["AngleAlice(deg)"]])
    deg_bob = np.array([df["AngleBob(deg)"]])
    n_A0B0 = np.array([df["A0B0"]])
    n_A1B0 = np.array([df["A1B0"]])
    n_A0B1 = np.array([df["A0B1"]])
    n_A1B1 = np.array([df["A1B1"]])

    corr = (n_A0B0 + n_A1B1 - n_A1B0 - n_A0B1) / (n_A0B0 + n_A1B1 + n_A1B0 + n_A0B1)
    s_par = corr[0][0] - corr[0][1] + corr[0][2] + corr[0][3]

    A = n_A0B0 + n_A1B1
    B = n_A1B0 + n_A0B1
    N = n_A0B0 + n_A1B1 + n_A1B0 + n_A0B1

    sigma_E = 2 * np.sqrt(A * B / N**3)

    sigma_s = np.sqrt(sum(sigma_E[0][i]**2 for i in range(4)))
    return ((np.round(corr, ROUND_TO), np.round(sigma_E, ROUND_TO)), (np.round(s_par, ROUND_TO), np.round(sigma_s, ROUND_TO)))

### ---- Curve fitting Functions ---- ##
#
def func_sin_fitt(x, a, b, c, d):
    return a * np.sin(b * (x + c)) + d

def estimate_sin_p0(x, y, n_points=None)

```

```

:
    """Estimate [a, b, c, d] starting values for a*sin(b*(x+c))+d
    from the max/min within the first n_points of (x, y)."""
    x = np.asarray(x)
    y = np.asarray(y)
    if n_points is not None:
        x, y = x[:n_points], y[:n_points]

    idx_max = np.argmax(y)
    idx_min = np.argmin(y)
    x_max, x_min = x[idx_max], x[idx_min]

    y_max, y_min = y[idx_max], y[idx_min]

    print(x_max, x_min, y_max, y_min)

    a0 = (y_max - y_min) / 2
    b0 = np.pi / np.abs((x_max - x_min))
    c0 = -(x_max + x_min) / 2 # note the sign fix
    d0 = (y_max + y_min) / 2

    return [a0, b0, c0, d0]

err_DEGREE = 0.5

columns = [
    'AngleAlice(deg)',
    'AngleBob(deg)',
    'A0B0',
    'A1B0',
    'A0B1',
    'A1B1',
]

data_bob = pd.read_csv(
    filepath_or_buffer = DATA_KORR_PATH
/ "2026-07-20_17-03-32_Verschraenkung_Bob.txt",
    sep = ';',
    comment = '#',
    header = None,
    names = columns
)
# data_bob.sort_values('AngleBob(deg)',
inplace=True, ignore_index=True)
data_alice = pd.read_csv(
    filepath_or_buffer = DATA_KORR_PATH
/ "2026-07-20_17-03-38_Verschraenkung_Alice.txt",
    sep = ';',
    comment = '#',
    header = None,
    names = columns
)
# data_alice.sort_values('AngleAlice(deg)',
inplace=True, ignore_index=True)
# print(data)

s_corr_vals = (get_correlation(data_bob)
)
print(
    f"S Value und Zählraten Bob\n",
    f"AB {s_corr_vals[0][0][0][0]} +/"

```

```

- {s_corr_vals[0][1][0][0]}    \n",
    f" A'B   {s_corr_vals[0][0][0][1]} +/
- {s_corr_vals[0][1][0][1]}    \n",
    f" AB'   {s_corr_vals[0][0][0][2]} +/
- {s_corr_vals[0][1][0][2]}    \n",
    f" A'B'  {s_corr_vals[0][0][0][3]} +/
- {s_corr_vals[0][1][0][3]}    \n",
    f" S-Value {s_corr_vals[1][0]} +/- {
s_corr_vals[1][1]}    \n",
)
s_corr_vals = (get_correlation(data_alice))
print(
    f"S Value und Zählraten Alice\n",
    f" AB   {s_corr_vals[0][0][0][0]} +/
- {s_corr_vals[0][1][0][0]}    \n",
    f" A'B  {s_corr_vals[0][0][0][1]} +/
- {s_corr_vals[0][1][0][1]}    \n",
    f" AB'  {s_corr_vals[0][0][0][2]} +/
- {s_corr_vals[0][1][0][2]}    \n",
    f" A'B' {s_corr_vals[0][0][0][3]} +/
- {s_corr_vals[0][1][0][3]}    \n",
    f" S-Value {s_corr_vals[1][0]} +/- {
s_corr_vals[1][1]}",
)
# print(get_correlation(data_alice))

```